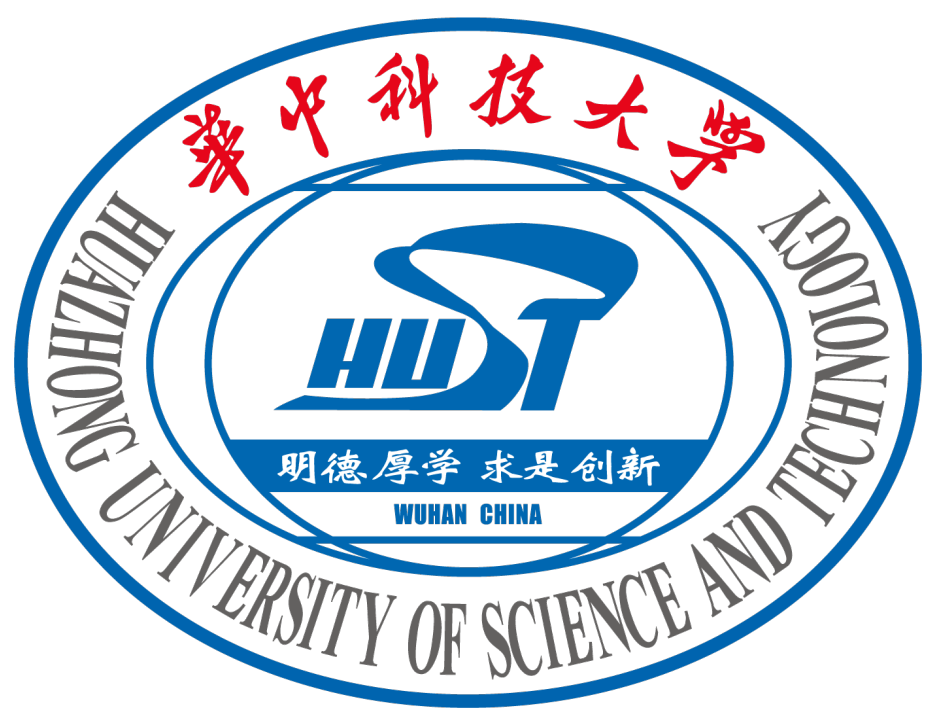




IGNN-Solver: A Graph Neural Solver for Implicit Graph Neural Networks



Junchao Lin¹, Zenan Ling^{1†}, Zhanbo Feng², Jingwen Xu³, Minxuan Liao¹, Feng Zhou⁴, Tianqi Hou⁵, Zhenyu Liao¹, Robert C. Qiu¹

¹School of EIC, Huazhong University of Science and Technology ²Shanghai Jiao Tong University ³Wuhan University of Technology
⁴Renmin University of China ⁵Lagrange Mathematics and Computing Research Center, Huawei, Paris

IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI), 2026
Graph-aware solver trajectories enable faster, scalable implicit GNN inference.

Introduction

Infinite depth, expensive inference. An implicit GNN represents node features through an equilibrium:

$$Z^* = f_\theta(Z^*, \hat{A}, X) = \sigma(WZ^*\hat{A} + b_\Omega(X)).$$

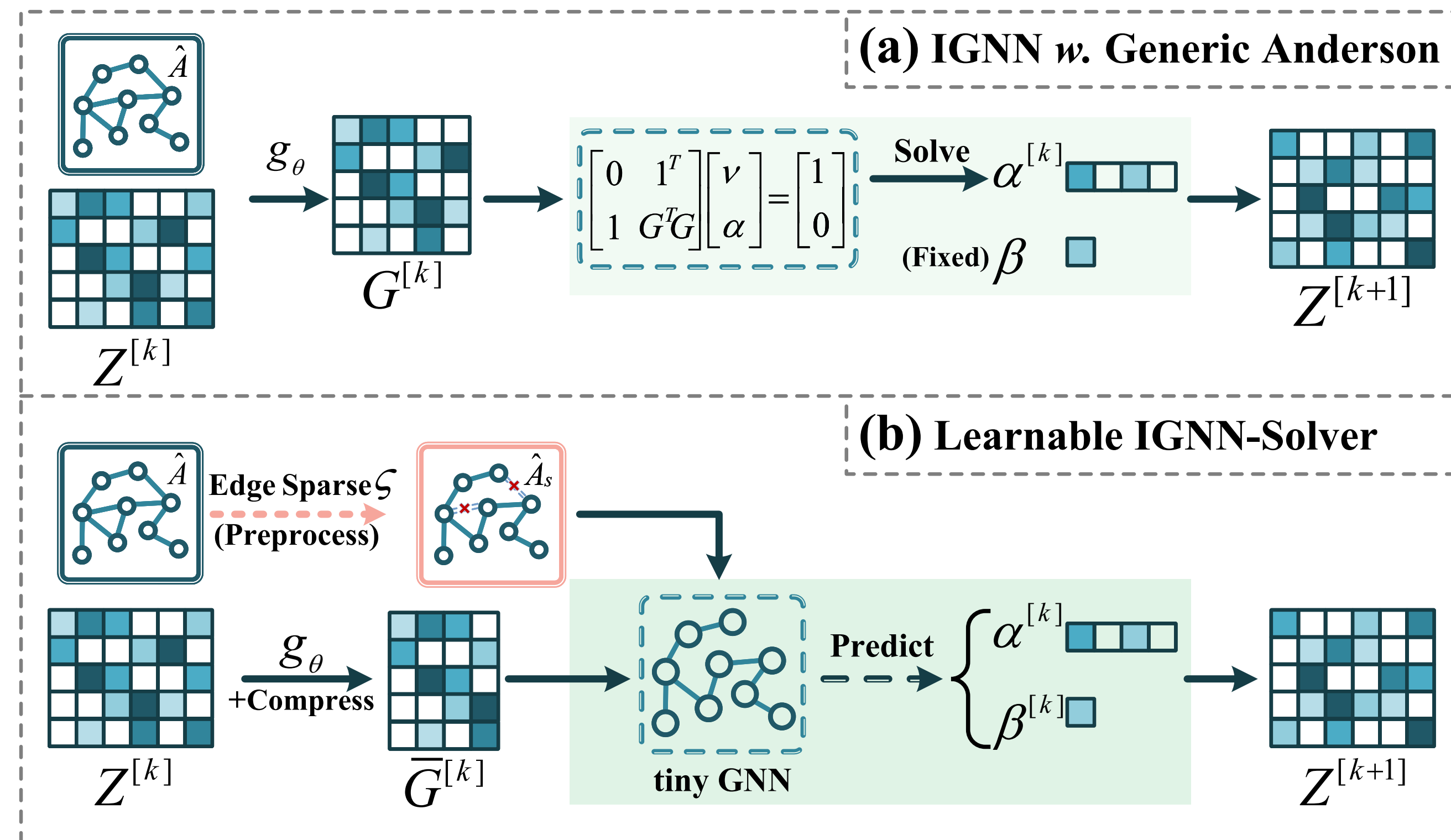
The global receptive field captures long-range dependencies. Repeated fixed-point iterations limit inference speed and scalability.

IGNN-Solver in one sentence. A tiny graph neural network learns how to accelerate the fixed-point trajectory, using graph structure and residual history.

Contributions.

- Learned initialization and graph-aware Anderson updates.
- Sparsification and history compression for large graphs.
- Evaluation up to **111M nodes and 1.6B edges**.

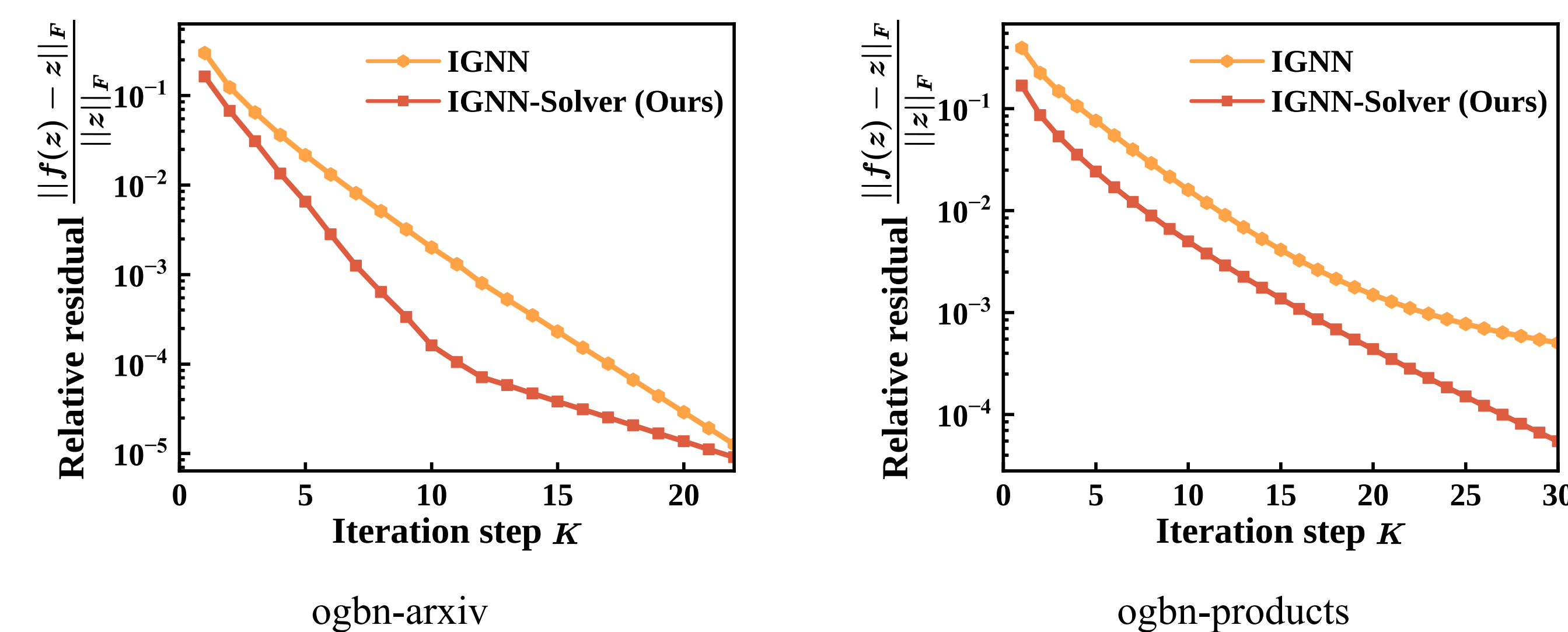
Graph-Aware Solver Pipeline



Generic AA versus IGNN-Solver. The tiny GNN predicts mixing coefficients from compressed residuals and a sparse auxiliary graph. (Original paper, Fig. 1.)

- **Initialize:** $Z^{[0]} = h_\phi(X)$.
- **Predict:** adapt coefficients to graph context and residual history.
- **Refine:** update with the original IGNN operator.

Faster Residual Reduction



Convergence beyond training unrolls. With $K = 6$ training steps, the learned solver generalizes to longer inference trajectories and reduces relative residuals faster. (Original paper, Fig. 5.)

Learnable Anderson Updates

From local fitting to graph-conditioned prediction. A tiny GNN uses residual history $G^{[k]}$ and graph context to predict $\alpha^{[k]}$ and $\beta^{[k]}$.

$$Z^{[k+1]} = \beta^{[k]} \sum_{i=0}^{m_k} \alpha_i^{[k]} f_\theta(Z^{[k-m_k+i]}) + (1 - \beta^{[k]}) \sum_{i=0}^{m_k} \alpha_i^{[k]} Z^{[k-m_k+i]}.$$

Structure and guarantee. Softmax weights sum to one; sigmoid bounds the mixing parameter. Under the paper's Cayley parameterization and coefficient constraints, a unique equilibrium and convergence are guaranteed.

Training the Solver

Freeze the IGNN; learn the solver. Unroll the solver for K steps and optimize its initializer and tiny GNN against numerical equilibrium targets.

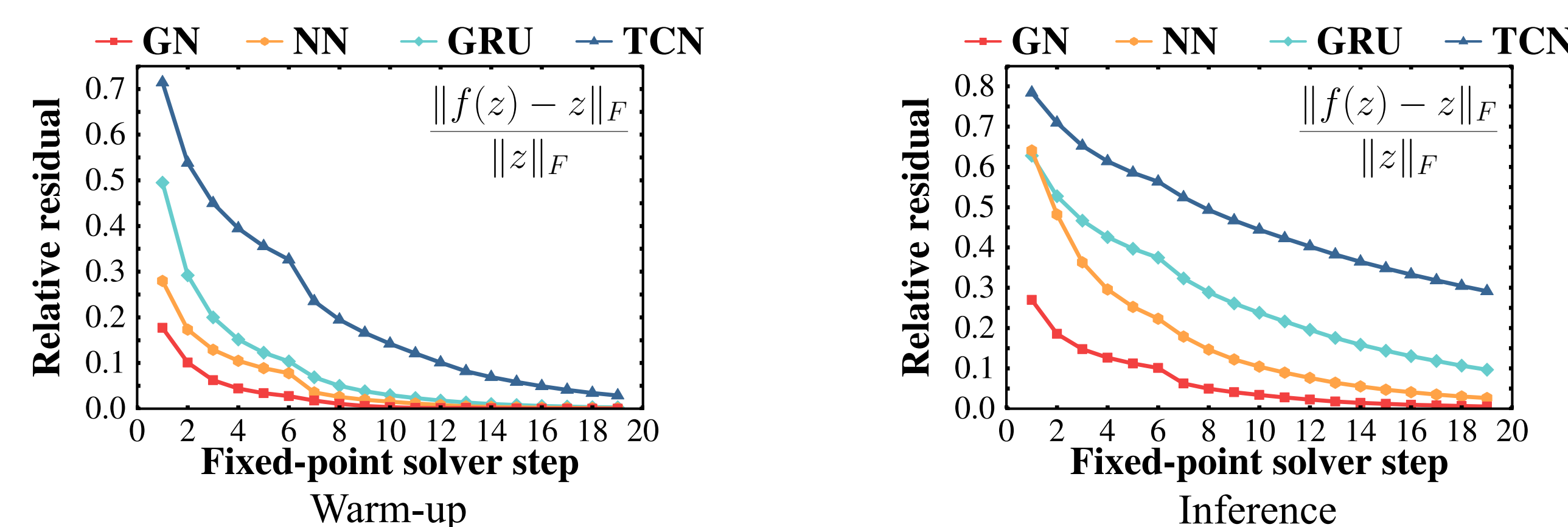
- **Convergence loss:** align intermediate iterates with the equilibrium; later steps receive larger weights.
- **Coefficient loss:** regularize the mixing rule using residual history.
- **Initializer loss:** improve the starting state.

The solver training phase requires **no class labels**.

Scaling with a Lightweight Auxiliary Graph

- **Sparsify** the auxiliary graph for cheaper coefficient prediction.
- **Compress** residual history with an MLP.
- **Keep the full graph** in the core IGNN fixed-point operator.

Why Graph Information Matters



Graph neural (GN) prediction converges fastest compared with NN, GRU and TCN alternatives on Citeseer. (Original paper, Fig. 7.)

Training Cost and Amortization

Compact solver module. Its independent training accounts for **about 1%** of overall training time in the paper's module-cost study.

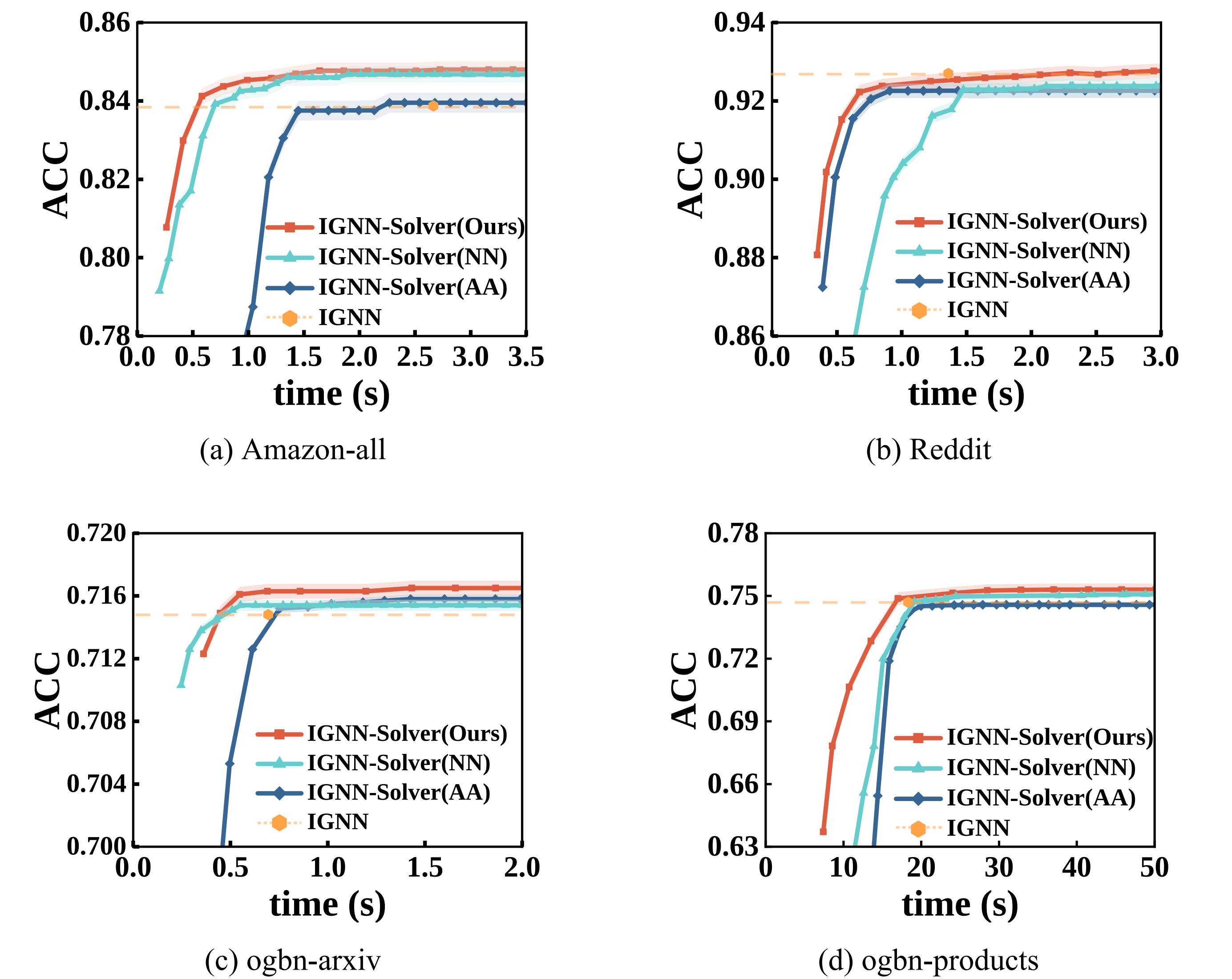
Joint training is a separate cost. Training time per epoch (Table V):

Dataset	IGNN (s)	Ours (s)
ogbn-arxiv	1.25	1.37
ogbn-products	84.20	88.51
ogbn-papers100M	1,245.3	1,296.8

(Original paper, Fig. 6 and Table V.) The module-only training share and joint per-epoch overhead use different accounting.

Speed–Accuracy Tradeoffs

1.5–8× faster at matched accuracy. The paper compares time at the IGNN base-line's accuracy on Pareto curves. Amazon-all reaches **8×** acceleration.



Original speed–accuracy curves from Fig. 2. Error bars reflect five independent runs; iteration budgets vary along each solver curve.

Node Classification Accuracy

Dataset	IGNN accuracy (%)	Ours accuracy (%)
Amazon-all	83.90 ± 0.51	84.50 ± 0.70
Reddit	92.30 ± 1.55	93.91 ± 0.31
ogbn-arxiv	70.49 ± 0.75	72.53 ± 0.41
ogbn-products	74.63 ± 0.24	74.90 ± 0.20

Selected results from Table I: mean ± standard deviation over five trials. Table results and matched-accuracy comparisons are separate.

Billion-Edge Graph: ogbn-papers100M

111M nodes; 1.6B edges. At the operating point reported in Table II:

Method	Accuracy (%)	Time (s)	Params
IGNN	65.11 ± 0.55	227.69	1.49M
IGNN + AA	65.07 ± 0.71	181.48	1.49M
IGNN-Solver	63.06 ± 0.23	36.24	1.50M

6.28× faster inference, with a **2.05 percentage-point accuracy reduction** relative to IGNN at this tabulated point.

Peak GPU memory: 12.26 GB (IGNN) and 13.44 GB (ours), from Table V. This operating point illustrates the efficiency–accuracy tradeoff at ultra scale.

†Correspondence: lingzenan@hust.edu.cn